Draw It or Lose It

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 4
Evaluation 4
Recommendations 7

Document Revision History

Version	Date	Author	Comments
1.0	09/20/25	Noelia Payero	Initial draft of software design document.
1.1	10/05/25	Noelia Payero	Revision of software design document.
1.2	10/19/25	Noelia Payero	Added Recommendations section addressing OS, storage, memory, distributed systems, and security

Executive Summary

Creative Technology Solutions (CTS) will develop a web-based version of *Draw It or Lose It* to allow multiple teams and players to compete across platforms. The game requires unique names for games and teams and only one instance of the game service in memory. The proposed solution uses a Java application with an Entity base class for shared attributes, the singleton pattern for GameService to ensure a single instance, and the iterator pattern to enforce name uniqueness. This design ensures scalability, maintainability, and future extensibility while efficiently transitioning the game from Android to web.
This project also evaluates deployment across multiple platforms, including Linux, Mac, Windows, and mobile devices. Based on characteristics, advantages, and weaknesses, Linux is recommended for server hosting due to its stability, security, and scalability. The game will be delivered via responsive web clients to ensure cross-platform compatibility, and development will utilize modern web tools with the appropriate team resources to support multiple desktop and mobile platforms.

Requirements

The client requires a web-based version of *Draw It or Lose It* supporting multiple teams and players per game. Each game and team must have a unique name to prevent conflicts. Only one instance of GameService should exist in memory. The game consists of four timed rounds, with drawings rendered from a stock library and fully displayed at the 30-second mark. The system must be cross-platform, maintainable, handle multiple users simultaneously, and allow future expansions such as additional game modes or chat. Development will use cross-platform web technologies (HTML, CSS, JavaScript) with IDEs such as Eclipse, IntelliJ, or VS Code. Additional resources may be needed for iOS/Android testing and deployment to ensure compatibility across all client platforms.

Design Constraints

The game must run in a web-based distributed environment, supporting multiple users and teams simultaneously. Each game, team, and player must have a unique identifier, and only one instance of the GameService can exist in memory, requiring the singleton pattern. The iterator pattern must be used to check for duplicate names before adding new entities. Performance constraints include rendering drawings smoothly within 30 seconds per round.

Additional constraints include ensuring scalability for future features, such as chat or multiple game modes, which require modular and loosely coupled design. The game must maintain cross-browser compatibility on desktops and mobile devices, ensuring consistent behavior across different platforms. Real-time synchronization is critical to maintain consistent game state when multiple players interact simultaneously. The system should handle offline or poor connections gracefully, caching actions or allowing reconnections. The drawing library must be extensible to support a growing number of images without overloading memory or network resources. Finally, accessibility considerations must be included, such as high-contrast graphics and keyboard navigation, to ensure all users can participate effectively.

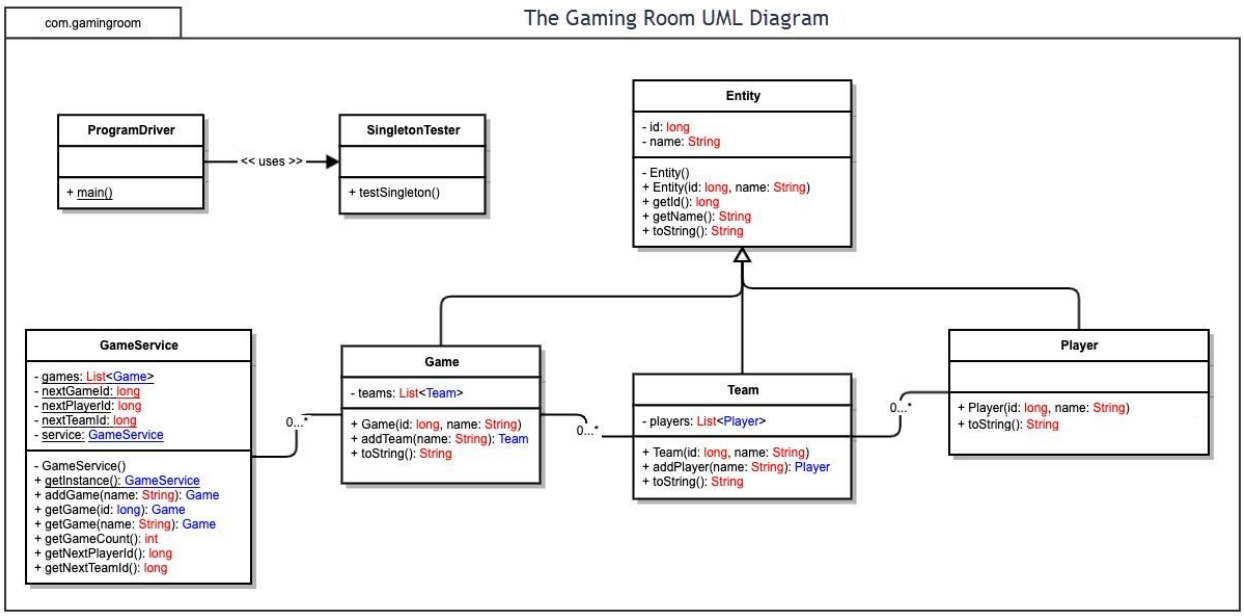
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical

components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML diagram shows the structure and relationships of the Draw It or Lose It application. The Entity class is a base class for Game, Team, and Player, providing shared attributes id and name, demonstrating inheritance.
GameService is a singleton that manages all Game instances, tracks unique IDs, and provides methods to add and retrieve games, illustrating the singleton design pattern. Game contains multiple Team objects, and Team contains multiple Player objects, showing composition, where each object is made up of other objects to model real-world relationships.
The design also uses encapsulation, keeping data private within classes and providing controlled access through public methods. Modularity is demonstrated by separating responsibilities across classes, making the system easier to maintain and extend. The iterator pattern ensures unique names when adding games, teams, or players.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Reliable and easy for development, but less common for hosting production servers. Hardware is expensive, limited scalability. macOS requires Apple hardware; no general cloud server images.	Stable, secure, and efficient; ideal for hosting web applications with minimal resource use. Open-source tooling, excellent support for containers (Docker/Kubernetes). Licensing is free for community distros; paid subscription for enterprise (RHEL/SLES).	Easy to set up, widely supported, familiar GUI, supports IIS/.NET. Windows Server per-core plus possible CALs. Weaknesses: higher cost, heavier resources, slower scaling for large deployments.	Not suitable for hosting; intended only for client access. Developer account only ($25 Android, $99 iOS). Cannot host production servers.
Client Side	Development for Mac clients requires ensuring cross-browser compatibility and testing on different macOS versions. Costs may be higher due to Apple hardware requirements. Time investment is moderate, as most modern web tools are supported, and expertise in macOS and web development is needed.	Linux clients require testing across different distributions and desktop environments. Costs are low since most tools are open-source, but additional time may be needed for configuration and troubleshooting. Expertise in Linux environments and web development is necessary.	Windows development is widely supported with broad browser compatibility, making setup easier. Costs are moderate (OS licensing may apply), time is reasonable due to available tools, and expertise in Windows environments and web technologies is required.	Supporting mobile clients requires responsive design and testing across devices and operating systems (iOS, Android). Costs can be moderate to high due to multiple device testing. Development time is significant, and expertise in mobile design, touch interaction, and cross-platform testing is critical.

Development Tools	Eclipse, IntelliJ, VS Code; HTML/CSS/JS; browser testing tools; Xcode for iOS builds. Cost: mostly free, except Xcode requires Apple hardware.	Java IDEs (Eclipse, IntelliJ), web development tools (HTML, CSS, JavaScript), command-line utilities for server management, and browser testing for cross-platform compatibility; container tools (Docker, Kubernetes). Mostly free/open-source.	Java IDEs (Eclipse, IntelliJ), web development tools, GUI-based development utilities, and browser testing tools to ensure consistent behavior across platforms; may include paid enterprise IDE licenses.	Responsive frameworks (Bootstrap, Tailwind), JS libraries for mobile (React, Vue); emulators; browser/device testing; optional wrappers (Capacitor/Ionic). Licensing includes free frameworks; paid if using enterprise CI/CD or macOS build machines.

Recommendations

1. **Operating Platform**: Linux is recommended for hosting *Draw It or Lose It* due to its stability, security, and efficiency. It supports scalable web applications, allows the game to expand to other platforms with minimal configuration, and can be supported by a development team using open-source and cross-platform tools. For the client, this means smooth multiplayer gameplay, simplified updates, and consistent experience across all supported platforms.

2. **Operating Systems Architectures**: The application will use a client-server architecture. The server runs on Linux, managing game logic, teams, and players, while clients access the game via web browsers on desktops and mobile devices. This ensures separation of concerns, easier maintenance, consistent gameplay, and allows the development team to build and test cross-platform clients efficiently.

3. **Storage Management**: In-memory lists are used to store games, teams, and players for fast access. This can later be extended to a relational database or NoSQL system if persistent storage is needed, allowing for scalability and data integrity. This approach ensures rapid gameplay response for users while giving The Gaming Room flexibility to track player progress, game history, and support future features like additional game modes or analytics.

4. **Memory Management**: The singleton pattern ensures only one instance of GameService exists, preventing duplicate data in memory. Java's garbage collection automatically frees unused objects, reducing memory leaks and improving performance. This guarantees that game state remains consistent for all players, preventing conflicts or crashes during multiplayer rounds, which is critical for a smooth user experience.

5. **Distributed Systems and Networks**: The game communicates across multiple platforms using HTTP or WebSocket protocols. The server synchronizes game state for all clients, and error handling ensures smooth recovery from network outages. Future enhancements, such as load balancing, can support high concurrency and large numbers of simultaneous players. This enables The Gaming Room to provide a reliable multiplayer experience across locations and devices, ensuring users can play together seamlessly even with network fluctuations.

6. **Security**: User information is protected using HTTPS for encrypted communication. Input validation prevents malicious data, and access to administrative functions is controlled. Linux allows extensive customization of security settings, giving us flexibility to implement firewalls, permissions, and monitoring tools tailored to the game. Combined with proper server configuration, these features help safeguard data across all platforms. This protects player information, maintains game integrity, and builds user trust, which is essential as the game expands to more platforms and players.

7. **Development Tools:** Development can be managed by a web team using IDEs such as Eclipse, IntelliJ, or VS Code, along with HTML, CSS, and JavaScript for client-side interfaces. Additional resources are recommended for iOS and Android testing and deployment to ensure compatibility across mobile platforms. Responsive frameworks (e.g., Bootstrap, Tailwind) and JavaScript libraries (React, Vue) support cross-platform web design, while emulators and browser/device testing ensure consistent behavior. Most tools are free or open-source, though Apple hardware is required for iOS builds, and some enterprise tools may carry licensing costs.

This ensures The Gaming Room's development team can efficiently create and maintain a cross-platform game experience, reducing development time while supporting a growing user base.